Proyecto 1 Etapa 2 - Analítica de textos

Juan Miguel Manrique – 202021447

Sebastián Contreras Salazar - 202020903

Andrés Felipe Gómez García - 202021189

Aumentación de Datos y Reentrenamiento del Modelo

El mejor modelo de la primera etapa del proyecto utilizaba el algoritmo de Naive-Bayes para la clasificación. Para esta segunda etapa, se decidió, para la aumentación de datos y posterior reentrenamiento del modelo, realizar las estrategias siguientes:

- Aumentación del segundo conjunto de datos mediante técnicas de prompting, esto con el objetivo de balancear las clases aumentando únicamente la clase minoritaria.
- Combinación entre el conjunto de datos correspondiente a la primera etapa del proyecto, con el conjunto de datos nuevo aumentado. Esto con el objetivo de incrementar la cantidad de datos totales.
- Volver a aumentar el conjunto de datos resultante para balancear una vez más las clases.
- Reentrenar el modelo con este nuevo conjunto de datos combinado y aumentado.

Para comparar el modelo original, con el modelo reentrenado, es necesario calcular las métricas con un conjunto de datos de prueba que no haya sido utilizado para el entrenamiento de ambos modelos, esto para evitar posibles sesgos.

Para ello, decidimos reservar una partición del segundo conjunto de datos aumentado, que fue excluido del reentrenamiento del modelo. Dado que dicha partición no fue utilizada para entrenar el modelo original y el segundo modelo, podrá ser utilizada para comparar las métricas sin ningún tipo de sesgo.

La implementación en código se encuentra en el archivo: ‘proyecto1_etapa2.ipynb’ en el repositorio del proyecto.

Las métricas obtenidas para ambos modelos fueron las siguientes:

```
------------------ Métricas en test final combinado ------------------
accuracy: 0.96
balanced_acc: 0.963
f1_macro: 0.958
f1_weighted: 0.96

Reporte por clase:
              precision    recall  f1-score   support

           1      1.000     1.000     1.000         9
           3      0.875     1.000     0.933         7
           4      1.000     0.889     0.941         9

    accuracy                          0.960        25
   macro avg      0.958     0.963     0.958        25
weighted avg      0.965     0.960     0.960        25


<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1e7c6d489d0>
```

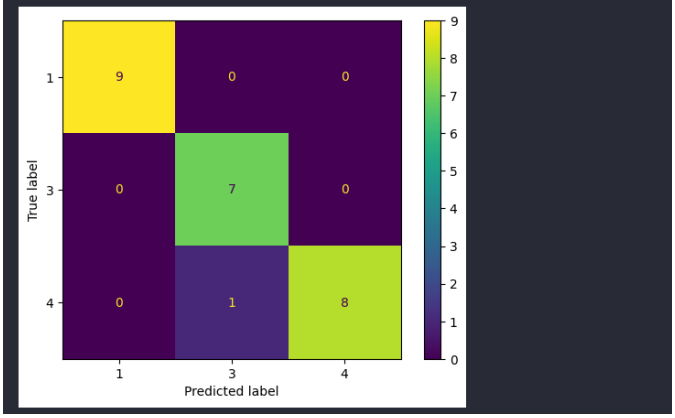


Fig 1. Metricas para modelo reentrenado

```
------------------ Métricas en test (modelo anterior) ------------------
accuracy: 0.84
balanced_acc: 0.852
f1_macro: 0.837
f1_weighted: 0.839

Reporte por clase:
              precision    recall  f1-score   support

           1      1.000     0.667     0.800         9
           3      0.700     1.000     0.824         7
           4      0.889     0.889     0.889         9

    accuracy                          0.840        25
   macro avg      0.863     0.852     0.837        25
weighted avg      0.876     0.840     0.839        25


<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1e7c6cf2e00>
```

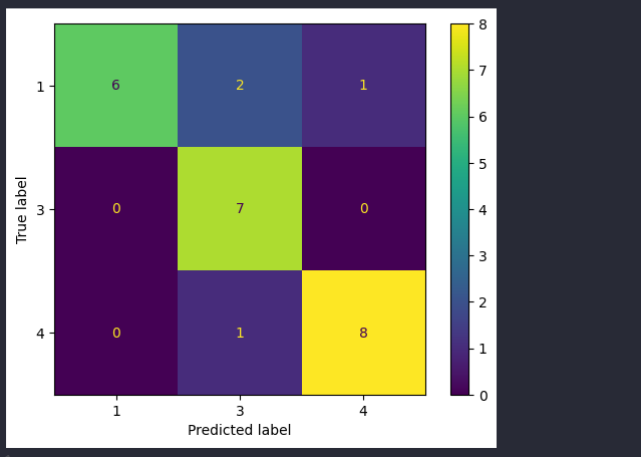


Fig 2. Métricas para el modelo original

Como resumen, se presenta la siguiente tabla comparativa:

Modelo	n_test	Accuracy	Balanced Acc	F1 Macro	F1 Weighted	Precisión Macro
NB original	25	0.84	0.852	0.837	0.839	0.863
NB reentrenado	25	0.96	0.963	0.958	0.960	0.958
Δ (reentrenado - orig.)	0	0.12	0.111	0.121	0.121	0.095

Hubo una mejoría del 0.121 en el f1-macro tras el reentrenamiento para el conjunto de datos seleccionado.

Proceso de Automatización del Proceso de Preparación de Datos, Construcción del Modelo, Persistencia del Modelo y Acceso por Medio de API

Para el proceso de automatización de la preparación de datos, se implementaron distintos transformadores que realizan el proceso de vectorización y normalización a textos crudos (los que recibirá el api por el endpoint de train). Dichos transformadores se incorporaron dentro del pipeline que fue exportado en un archivo .joblib que se encuentra en el repositorio.

Para la implementación del API, se hizo uso del framework FastAPI. Los dos endpoints que se definieron fueron predict, que será utilizado para clasificar textos dados por el usuario y retrain, que será utilizado para reentrenar el modelo con textos dados por el usuario.

FastAPI provee una interfaz gráfica que facilita probar los endpoints implementados, los resultados de estos se muestran a continuación:

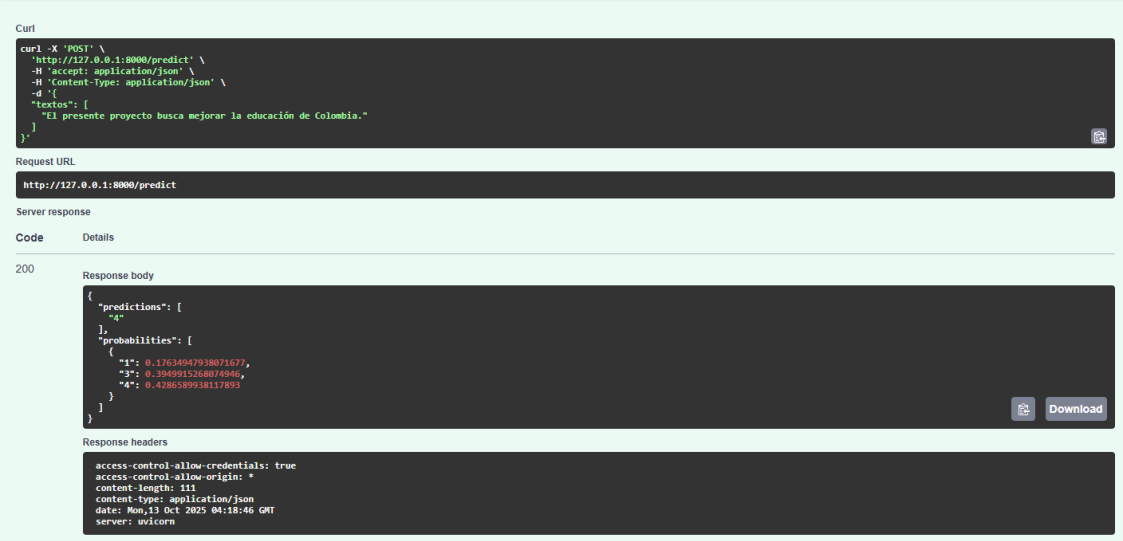


Fig 3. Ejemplo de ejecución endpoint /predict



Fig 4. Ejemplo de ejecución endpoint /retrain

Definiciones de reentrenamiento

1) Full-batch acumulativo (desde cero sobre histórico consolidado)

Definición: se vuelve a entrenar todo el pipeline (Normalizador → TF-IDF → MultinomialNB) desde pesos iniciales, pero con el histórico acumulado + el nuevo lote. Se excluye un holdout fijo (si existe) para evaluar sin fuga. Se versiona un nuevo .joblib y se actualiza el alias latest.joblib.

Ventajas: hay calidad y estabilidad, reincorpora todo el conocimiento previo y absorbe el lote nuevo sin depender del orden de llegada. Además, es reproducible y auditable. El dataset de entrenamiento queda documentado, hay versionado de artefactos y card de modelo.

Desventajas: alto costo y latencia, dado que reentrenar todas las etapas requiere más tiempo que una actualización parcial. Además, hay una necesidad de gestionar datos: persistir histórico, deduplicar y controlar el holdout para evitar fuga.

Cómo se materializa en nuestra API:

- artifacts/train_store.csv: histórico de (texto, label) que se acumula en cada /retrain.
- artifacts/eval_holdout.csv: conjunto de evaluación fijo (si no existe y hay muestras suficientes, se crea una única vez de forma estratificada).
- El endpoint /retrain: carga modelo vigente y el histórico, calcula métricas antes sobre el mismo set de evaluación, concatena histórico + lote nuevo, re-entrena desde cero el pipeline con ese conjunto, calcula métricas después en el mismo set, guarda nb_pipeline_v2_YYYYMMDD_HHMMSS.joblib y actualiza latest.joblib.

2) Actualización incremental (online) con partial_fit

Definición: mantener fijo el pipeline y actualizar sólo el clasificador con los nuevos ejemplos mediante partial_fit. Para texto, se congela el vectorizador o se usa HashingVectorizer para evitar mantener vocabulario.

Ventaja: muy rápido y barato puesto que permite absorber lotes pequeños de datos sin reentrenar todo y con mínima indisponibilidad del servicio.

Desventaja: riesgo de sesgo por orden y limitación del vocabulario, dado que, si cambian los términos, el TF-IDF congelado no incorpora nuevas palabras. No replantea hiperparámetros ni corrige desequilibrios históricos salvo que se diseñe una estrategia adicional.

3) Reentrenamiento acumulativo con ventana temporal o ponderación

Definición: reentrenar desde cero, pero sólo con una ventana reciente o ponderando los ejemplos nuevos con mayor peso y mezclándolos con una muestra del histórico. El pipeline se vuelve a ajustar, pero con menos datos o con énfasis en los más recientes.

Ventaja: equilibrio adaptación y costo: se adapta a concept drift dando prioridad a lo reciente sin requerir todo el histórico y con menor tiempo de ajuste que el full-batch.

Desventaja: puede perder cobertura en casos raros o sobre-adaptarse a picos temporales si los pesos no se calibran bien.

<u>Reentrenamiento elegido</u>

Para la Etapa 2 elegimos la estrategia 1) Full-batch acumulativo con holdout fijo:

Datos de entrenamiento: histórico de la Etapa 1 + datos nuevos de la Etapa 2 + (cuando aplicó) aumentación por prompting para la clase minoritaria.

Pipeline: TextNormalizer (NLTK) → TfidfVectorizer → MultinomialNB, todo en un único .joblib.

Automatización: /retrain acumula y deduplica, excluye el holdout fijo del fit, re-entrena desde cero el pipeline, versiona y expone métricas antes y después en el mismo set de evaluación.

Motivación: con el tamaño y complejidad del proyecto, esta opción maximizó calidad, estabilidad y trazabilidad. El costo computacional fue manejable y las métricas mostraron mejora sostenida frente al modelo anterior, validando la decisión.

Desarrollo de la Aplicación y Justificación

Es necesario describir al usuario/rol de la organización que va a utilizar la aplicación, el cual en este caso sería un analista de participación ciudadana y ODS en UNFPA (o entidad aliada de planeación). Igualmente, el proceso de negocio que lo apoyaría es Triage y análisis de opiniones ciudadanas, para priorizar intervenciones en ODS 1 (fin de la pobreza), ODS 3 (Salud y bienestar) y ODS 4 (educación de calidad). Aún

más, la preparaión de insumos para informes y tableros institucionales son parte del proceso de negocio que se apoya.

Por otro lado, la conexión entre esa aplicación y el proceso de negocio que se va a apoyar es que la aplicación recibe textos, clasifica automáticamente por ODS y devuelve probabilidades, donde los casos con baja confianza se marcan para revisión humana. Esto acelera el flujo de recolección, etiquetado, análisis y decisiones, estandariza criterios y mejora trazabilidad.

Ahora bien, con esto, es posible determinar la importancia que tiene para ese rol la existencia de esta aplicación, el cual es la reducción de tiempos de lectura/etiquetado manual, homogenización de criterios entre equipos y turnos, y permite retroalimentación al modelo mediante reentrenamientos controlados para mantener vigencia frente a nuevos temas o vocabularios.

Con esto, ya es posible desarrollar una aplicación web para interactuar con el resultado del modelo analítico a partir de un texto o textos dados por el usuario, incluyendo la predicción como la probabilidad asociada a la misma.

Una demostración de la aplicación desarrollada y construida se puede observar en el video de presentación adjunto en el repositorio.

Para desplegarla se tiene un readMe que contiene los datos necesarios para el despliegue de la página web de manera local.

Ahora bien, los recursos informáticos para entrenar, ejecutar, persistir el modelo y desplegar la aplicación, necesita mínimo para entrenar y reentrenar un CPU de 2-4 vCPU, con un RAM de 8-16 GB mínimo. Esto se debe a que el pipeline usa vectorización clásica y modelos tradicionales que corren en segundos/minutos, por lo que no se requiere un GPY. Igualmente, el retrain debe devolver precisión, recall y F1 y actualizar la versión binaria del modelo.

Para ejecutar se necesita 1-2 vCPU de CPU y 1-2 GB de RAM mínimo. Igualmente, se necesita 5-10 req/s con textos cortos para un throughput esperado. Igualmente se necesita una API REST para ejecutar, donde se usa POST /predict que recibe n instancias y devuelve la lista de predicciones en el mismo orden. Igualmente, usa POST /retrain el cual recibe instancias con variable objetivo y retorna la precisión, recall y F1. Aún más, se necesita un formato de intercambio JSON que respeta el esquema del CSV original.

Por otro lado, para el despliegue de la página web local en Docker, se necesita una infraestructura mínima de 1 VM (2vCPU, 4 GB RAM, 20 GB disco), y se necesita correr estos comandos para desplegarlo desde la terminal:

1. docker compose build --no-cache api
2. docker compose up –d
3. docker compose ps

Si se quiere verificar el estado se usa este comando:

1. curl http://localhost:8000/health

Ahora bien, para probar los endpoint como /predict (devuelve la lista en el mismo orden de entrada) se usa el siguiente comando:

1. curl -X POST "http://localhost:8000/predict" -H "Content-Type: application/json" --data-binary @api-test-data/predict.json

Para /retrain se usa el siguiente comando:

1. curl -X POST "http://localhost:8000/retrain" -H "Content-Type: application/json" --data-binary @api-test-data/retrain.json

Si se desean ver los logs se usa el siguiente comando:

1. docker logs ods-api –f

Y para detener todo se usa el sigueinte comando:

1. docker compose down

En otro lado, la aplicación es usada por el Analista de participación/ODS de la entidad como la UNFPA. El analista interactúa con una web interna que permite: (i) clasificar textos y ver predicción + probabilidad; y (ii) reentrenar el modelo cuando hay nuevos ejemplos validados. Por otro lado, el flujo de proceso se divide en 4 partes: la primera es la recolección y filtrado donde equipos operativos capturan opiniones y filtran según el contexto del proyecto (ODS 1,3 y 4). Segundo es la clasificación asistida donde el analista carga textos en “Clasificar” para obtener etiqueta y probabilidad por instancia. Tercero es la revisión humana con los casos con baja confianza. Cuarto es la mejora continua donde el analista consolida ejemplos válidos y ejecuta “Reentrenar” la API devolviendo la precisión, recall y F1, y se actualiza versión binaria del modelo para inferencias futuras.

En cuanto la integración técnica y acceso, se propone en la topología que este el contendor de Docker en una MV interna con la IP privada. Si la organización adquiere un dominio, se usa split-DNS para que el dominio resuelva a la IP solo dentro de la red específicamente. Para la publicación y seguridad se podría tener detrás de reverse proxy con SSO/LDAP y allow-list por rangos IP o VPN corporativa. Igualmente, se podría tener áreas que puedan enviar lotes al endpoint de predicción y leer los resultados, donde la capa de analítica registra logs para auditoría.

Los principales riesgos para el usuario final y sus mitigaciones son los siguientes: un error de clasificación que sesgue decisiones se reduce mostrando la probabilidad de cada predicción, aplicando un umbral de baja confianza que active revisión humana, y habilitando el reentrenamiento con reporte de Precisión, Recall y F1 en cada ciclo; el desbalance de clases y el data drift ante nuevos temas o vocabulario se gestionan con aumentación por prompting en clases minoritarias, muestreo estratificado y reentrenamientos periódicos con monitoreo de métricas; los riesgos de privacidad y seguridad al tratar textos con datos personales se mitigan mediante anonimización previa, cifrado en tránsito (TLS) y controles de acceso; la disponibilidad y latencia en

picos o fallas del contenedor se mejoran con contenedores livianos, health-checks y escalado horizontal; las entradas mal formadas (JSON/XLSX fuera de esquema) se abordan con validación de esquema y mensajes de error guiados; y las versiones obsoletas del modelo se previenen registrando la versión del binario en logs y aplicando una política de reemplazo controlado tras cada reentrenamiento, con capacidad de rollback.

Resultados

Video de resultados: https://youtu.be/OnH4BahYGJs

Trabajo en Equipo

Roles y tareas:

- Líder de analítica y de proyecto (Juan Miguel Manrique): diseñó y afinó el pipeline, validó con GridSearchCV orientado a F1-macro, construyó matriz de confusión y tabla comparativa base/etapa 2, y dejó el mejor estimador listo para exportar. Igualmente, estuvo a cargo de la gestión del proyecto, definiendo las reuniones, y verifica las asignaciones de tareas para que la carga pueda ser equitativa.
- Ingeniero de datos (Sebastián Contreras Salazar): implementó aumentación por prompting enfocada en la clase minoritaria, consolidó dataset final, reentrenó el mejor NB sobre datos aumentados y versionó el artefacto con joblib.
- Ingeniero de software y desarrollador de aplicación (Andres Gómez): empaquetó el modelo como servicio API REST (FastAPI, contenedor Docker), expuso POST /predict, y POST /retrain, añadió logging básico y página web simple para interacción del usuario.

Horas dedicadas:

Sebastián Contreras Salazar: 17 h (modelado, validación, comparativos, documentación de métricas).

Juan Miguel Manrique: 26 h (aumentación, preparación de datos finales, reentrenamiento y exportación).

Andrés Gómez: 22 h (API, Docker, UI mínima, pruebas de integración).

Algoritmo de cada integrante;

Sebastián Contreras Salazar: Multinomial Naive Bayes con TF-IDF (selección y ajuste).

Juan Miguel Manrique: NB reentrenado con datos aumentados por prompting (mejoras en F1-macro).

Andrés Gómez: Servificación del pipeline NB + TF-IDF (no cambió el algoritmo, sí la orquestación).

Retos y cómo los resolvimos:

Desbalance de clases:

Los falsos negativos en la clase minoritaria afectaban F1-macro y por ende la mitigación: prompting dirigido para sintetizar ejemplos plausibles, curación ligera y reentrenamiento sobre el dataset extendido. Igualmente, se realizó una normalización de texto: variaciones ortográficas y ruido para la mitigación: TextNormalizer (limpieza, lowercasing, stopwords) antes de TF-IDF. También trazabilidad del artefacto: riesgo de mezclar versiones hacía mitigación: convención nb_pipeline_v2_<timestamp>.joblib + registro de métricas al reentrenar. Finalmente, interfaz mínima y probabilidades: requerimiento de exponer etiqueta + probabilidad hacía la mitigación: endpoint /predict retorna label, proba y top_terms (explicabilidad ligera).

Distribución de puntos:

Sebastián Contreras Salazar: 33

Juan Miguel Manrique: 34

Andrés Gómez: 33

Uso de ChatGPT:

Apoyo en esqueletos de código (controladores de API, validadores de esquema), redacción y revisión de texto técnico, y generación de ideas de prompts para aumentación (siempre validadas por el equipo). Todas las decisiones y pruebas fueron verificadas y documentadas por nosotros.

Puntos para mejorar para siguientes entregas:

1. Automatizar pruebas de regresión de métricas tras cada reentrenamiento.
2. Incorporar explicación del modelo a nivel de palabras clave/top-features en UI.
3. Añadir monitoreo continuo de data drift.
4. Conectar la página web con Inteligencia Artificial Generativa para aprender automáticamente de los datos introducidos nuevos.